Unit 3

Experiment 8 - Kafka-Based Event-Driven Publishing Pipeline

Aim:

Develop an event-driven publishing pipeline using Kafka that processes scheduled posts, ensures reliability with retries and dead-letter queues, and maintains idempotent operations.

Description:

Modern distributed applications require systems that are scalable, resilient, and capable of handling asynchronous workflows efficiently. Traditional synchronous architectures often lead to tight coupling, reduced scalability, and performance bottlenecks.

This experiment introduces the concept of **event-driven architecture (EDA)** using Apache Kafka to build a **publishing pipeline for scheduled posts,** incorporating **Docker-based infrastructure setup**, **reliability mechanisms**, and **distributed system design principles**. It focuses on decoupling services, enabling asynchronous communication, and ensuring system reliability through **retry mechanisms, dead-letter queues (DLQ),** and **idempotency patterns**.

By implementing this pipeline, students gain exposure to **real-world distributed system design principles**, commonly used in large-scale applications such as social media platforms, financial systems, and real-time analytics engines. Additionally, containerization using Docker enables easy setup, portability, and reproducibility of Kafka environments, making it highly suitable for development and testing.

Theoretical Explanation:

1. **Eveny-Driven Architecture**

   Event-driven architecture is a paradigm where components communicate via events instead of direct calls.

   Example of events: `post_created, payment_completed, post_liked`

   Why EDA?

   - Reduces coupling → Services don't depend on each other directly
   - Improves scalability → Each service can scale independently
   - Enables asynchronous processing → Tasks don't block each other

   How EDA works:

   - Service generated an event
   - Event is sent to a message system (Kafka)
   - Other services consume the event

   When to Use:

   - Real-time systems
   - Microservices communication
   - Background processing (e.g., scheduling posts)

   When NOT to Use:

- Simple CRUD systems
- Low-scale applications
- Projects without asynchronous needs

Academic Insight:

EDA aligns with distributed system design, enabling independent scalability and fault isolation.

2. Docker Fundamentals

Problem without Docker:

- "Works on my machine" issue (not working same on all machines)
- Different environments lead to errors

Docker: A tool that packages an application with all its dependencies into a container.

Analogy: A portable box that runs anywhere.

Virtual Machines vs Docker

Feature	**Virtual Machine**	**Docker**
Size	Heavy (GBs)	Lightweight (MBs)
Speed	Slow startup	Fast startup
OS	Full OS	Shares host OS

Image vs Container

Concept	**Meaning**
Image	Blueprint
Container	Running Instance

Why we use Docker here

- Simplifies Kafka Setup
- Avoids dependency issues
- Ensures reproducibility

When NOT to use Docker

- High-performance production systems without orchestration
- Systems needing direct hardware control

3. Apache Kafka Overview

Kafka is a **distributed event streaming platform** used for building real-time pipelines for sending and receiving messages or events.

Core Components:

- Producer → Sends messages
- Consumer → Receives messages
- Topic → Category of messages
- Broker → Kafka server

Why Kafka?

- High throughput → handles millions of messages efficiently
- Fault tolerance → data is replicated across brokers
- Horizontal scalability → can add more brokers easily

Pros:

- Durable messaging
- Scalable architecture
- Real-time processing

Cons:

- Complex setup → Requires configuration
- Requires monitoring → for tracking performance and health
- Higher operational overhead → needs maintenance

4. Docker for Kafka Deployment

Why Use Docker for Kafka?

- Simplifies setup (Kafka + Zookeeper)
- Eliminates environment conflicts
- Enables reproducible environments
- Ideal for development & testing

When to Use Docker:

- Local development
- Testing distributed systems
- CI/CD pipelines

Docker-Based Kafka Setup (KRaft Mode)

KRaft mode enables kafka to run **without Zookeeper** using Raft protocol.

```
version: '3'
services:
    kafka:
            image: apache/kafka:latest
            container_name: kafka
            ports:
                    - "9092:9092"
            environment:
                    KAFKA_NODE_ID: 1
                    KAFKA_PROCESS_ROLES: broker,controller
                    KAFKA_LISTENERS: PLAINTEXT://:9092,CONTROLLER://:9093
```

```
KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
KAFKA_CONTROLLER_QUORUM_VOTERS: 1@localhost:9093
KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
KAFKA_LOG_DIRS: /tmp/kraft-combined-logs
```

How it works

- Single container runs broker + controller
- No Zookeeper dependency
- Uses Raft protocol internally

Docker-Based Kafka Setup (ZooKeeper Mode)

```
version: '3'
services:
  zookeeper:
    image: confluentinc/cp-zookeeper:latest
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181

  kafka:
    image: confluentinc/cp-kafka:latest
    ports:
      - "9092:9092"
    environment:
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
```

How It Works:

- Zookeeper manages Kafka metadata
- Kafka broker handles messaging
- Containers run isolated services

Academic Insight:

Docker supports containerized distributed systems, improving deployment efficiency and portability.

5. Producer-Consumer Model

Concept

- Producer → sends event
- Consumer → processes event

Producer Example

```
kafkaTemplate.send("post-topic", postEvent);
```

Consumer Example

```
@KafkaListener(topics = "post-topic")
public void consume(PostEvent event) {
    processPost(event);
}
```

Why This Model?

- Decouples services
- Enables asynchronous execution
- Improves system flexibility

6. Scheduler + Event Publishing

Flow:

- Scheduler checks due posts
- Generates event
- Sends event to Kafka
- Consumer processes publishing

Why Scheduler needed?

- Automates delayed execution
- Ensures time-based processing

7. Retry Strategies & Exponential Backoff

Why Needed?

- Handle transient failures (network issues, temporary service downtime, etc.)
- Avoid immediate retries causing overload

Exponential Backoff: Retries with increasing delay – 1s, 2s, 4s, 8s, 16s,...

Example

```
int retry = 0;
while(retry < MAX_RETRIES) {
    try {
        process(event);
        break;
    } catch(Exception e) {
        retry++;
        Thread.sleep((long)Math.pow(2, retry) * 1000);
    }
}
```

Why it Works

Benefit	**Explanation**
Prevents overload	Gives system recovery time
Handles temporary failures	Many failures resolve automatically
Improves stability	Avoids retry storms

Limitation:

- Slower recovery
- Not ideal for real-time systems

Academic Insight:

Ensures fault tolerance and system resilience.

8. Dead Letter Queue (DLQ)

A separate queue for failed messages.

Why Use DLQ?

- Prevents blocking pipeline
- Enables debugging

Flow:

Failed Message → Retry → DLQ

Benefits

- Prevents pipeline blockage
- Enables debugging
- Keeps system running

Drawbacks

- Requires monitoring
- Failed messages may be ignores

9. Idempotency

Why Needed?

Kafka guarantees at-least-once delivery, so duplicates may occur.

Solution:

```
if(processedEvents.contains(eventId)) return;
```

Example: Clicking “pay” button twice should not charge twice.

Importance:

- Prevents duplicate actions
- Maintains data correctness

Academic Insight:

Ensures data consistency in distributed systems.

10. Distributed System Reliability

Key Concepts

- Fault Tolerance → System continues despite failures

- Message durability → data is not lost
- At-least-once delivery → message is delivered at least once

Experiment 8.1: Kafka Producer-Consumer Implementation

Conceptual Focus:

- Event-driven architecture
- Kafka (KRaft mode)
- Producer-consumer model
- Docker-based deployment

Implementation Overview:

- Setup Kafka using Docker (KRaft mode)
- Create scheduler for due posts
- Produce events to Kafka topic
- Build consumer to process events

Experiment 8.2: Reliability: Retry, DLQ & Idempotency

Conceptual Focus:

- Retry strategies
- Dead Letter Queue
- Idempotency
- Fault tolerance

Implementation Overview:

- Implement retry logic with exponential backoff
- Route failed messages to DLQ topic
- Track processed events
- Ensure duplicate-safe processing

Integration Perspective

Kafka in KRaft mode enables efficient asynchronous communication without external dependencies, while Docker ensures portability and ease of deployment. Combined with retry, DLQ, and idempotency, the system achieves high reliability, scalability, and consistency.

Assignments

Assignment 1 – Kafka Setup (Docker)

- Run Kafka using Apache image (KRaft mode)
- Create topic and verify setup

Assignment 2 – Producer & Consumer

- Send events from scheduler
- Consume and process events

Assignment 3 – Retry Mechanism

- Implement exponential backoff

Assignment 4 – DLQ

Route failed messages to DLQ topic

Assignment 5 – Idempotency

Prevent duplicate event processing

Key Takeaways

- Kafka enables scalable event-driven systems
- KRaft simplifies Kafka architecture
- Docker enables reproducible environments
- Retry and DLQ improve reliability
- Idempotency ensures correctness

Future Scope

These concepts can be extended to implement **real-time streaming pipelines, microservices communication, distributed tracing systems, and cloud-native event processing architectures using Kubernetes**.